

Text Large Model + YOLOv11 + Intelligent Garbage Classification

Function Description

After the program starts, give the garbage classification command to the large model. After thinking, the large model will control the robotic arm to sort the garbage blocks on the table.

Startup

After entering the docker container, enter the following command in the docker terminal:

```
ros2 launch largemodel largemodel_control.launch.py text_chat_mode:=True
```

Then open a second terminal and enter the following command:

```
ros2 run text_chat text_chat
```

Garbage classification has the following types of commands:

- Sort a single type of garbage: for example, sort toxic garbage
- Sort all garbage on the desktop: for example, clean up all garbage on the desktop

Sort Single Type of Garbage

Startup

Input in the text_chat terminal:

```
Clean up toxic garbage
```

Press Enter. For the voice version, after waking up the voice module, directly say "Clean up toxic garbage" to the voice module. Wait for the large model to think and reply, as shown below:

```
user input: Clean up toxic garbage
okay@, let me think for a moment... -[INFO] [1775128734.527912819] [text_chat_node]: Decision making AI planning:Call the garbage sorting function, with the parameter 'tox'.
user input: [INFO] [1775128736.821845069] [text_chat_node]: "action": ["garbage_sort('tox')"], "response": "I'm going to clean up the toxic garbage now, let me get started!"
```

The robotic arm will grip the toxic garbage label block on the table.

Task Planning

Call `garbage_sort('tox')` to sort toxic garbage.

Core Source Code Analysis

The `garbage_sort` function source code is located at:

```
LargeModel_ws/src/largemodel/largemodel/action_service.py
```

```
def garbage_sort(self, type_):
    Arm.Arm_serial_servo_write6(90,120,0,0,90,30,1000)
    time.sleep(1.0)
    type_ = type_.strip("'\"") # Remove single and double quotes
```

```

if type_ == "rec":
    target_type_ = float(1)
elif type_ == "tox":
    target_type_ = float(2)
elif type_ == "wet":
    target_type_ = float(3)
elif type_ == "dry":
    target_type_ = float(4)
else:
    self.get_logger().info(
        "Fatal ERROR:Incorrect type_ input,Does the AI output not meet expectations?"
    )
    self.action_status_pub(
        "garbage_sort_failed", type_=type_
    )
    return

cmd1 = "ros2 run largemodel_arm grasp"
cmd2 = f"ros2 run largemodel_arm yolov11_garbage --ros-args -p target_type:={
target_type_:}.1f}"

subprocess.Popen(
    [
        "gnome-terminal",
        "--title=grasp",
        "--",
        "bash",
        "-c",
        f"{cmd1}; exec bash",
    ]
)

subprocess.Popen(
    [
        "gnome-terminal",
        "--title=yolov11_garbage",
        "--",
        "bash",
        "-c",
        f"{cmd2}; exec bash",
    ]
)

```

Let's mainly analyze the garbage recognition program yolov11_garbage, source code path:

`LargeModel_ws/src/largemodel_arm/largemodel_arm/yolov11_garbage.py`

```

#Get the type of garbage to be sorted through command line parameters
self.declare_parameter('target_type', 1.0)
self.TargetType = int(self.get_parameter('target_type').get_parameter_value().double_value)
#According to the value of self.TargetType, modify self.waste, which contains all garbage
names of this type
if self.TargetType == 1:

```

```

        self.waste = ['Newspaper', 'Zip_top_can', 'Book', 'Old_school_bag']
    elif self.TargetType == 2:
        self.waste = ['Syringe', 'Expired_cosmetics', 'Used_batteries', 'Expired_tablets']
    elif self.TargetType == 3:
        self.waste = ['Fish_bone', 'Egg_shell', 'Apple_core', 'Watermelon_rind']
    elif self.TargetType == 4:
        self.waste = ['Toilet_paper', 'Peach_pit', 'Cigarette_butts', 'Disposable_chopsticks']

#Initialize a YOLO object detection model
model = YOLO("/home/yahboom/ultralytics/ultralytics/best.onnx")
#Create a subscriber to subscribe to the transformed image data topic
self.image_sub =
self.create_subscription(ImageMsg, "/image_data", self.image_sub_callback, qos_profile=1)

#In the callback function image_sub_callback
# Use YOLO11 for object detection
results = self.yolo_model(self.img, save=False, verbose=False)
boxes = results[0].boxes
#Iterate through the detection results
for box in boxes: # detections per image
    x_min, y_min, x_max, y_max = map(int, box.xyxy[0])
    class_id = int(box.cls)
    confidence = float(box.conf)
    label = f"{model.names[class_id]} {confidence:.2f}"
    # Calculate the center of mass position
    center_x = (x_min + x_max) // 2
    center_y = (y_min + y_max) // 2
    (a, b) = (round(((320 - center_x) / 4000), 5), round(((480 - center_y) / 3000) *
0.8+0.15, 5))
    res_name = model.names[class_id]
    print("res_name: ", res_name)
    if res_name in self.waste:
        print("+++++")
        self.cnt = self.cnt + 1
        self.detect_flag = True
    if self.pubPos_flag == True and res_name in self.waste:
        self.detect_flag = True
        self.pubPos_flag = False
        center = AprilTagInfo()
        center.x = b
        center.y = a
        center.z = 0.03
        #Determine garbage type
        if str(model.names[class_id]) in self.recyclable_waste:
            center.id = 1
        elif str(model.names[class_id]) in self.toxic_waste:
            center.id = 2
        elif str(model.names[class_id]) in self.wet_waste:
            center.id = 3
        elif str(model.names[class_id]) in self.dry_waste:
            center.id = 4
        print("center: ", center)
        #Publish garbage position information topic message

```

```

self.pos_info_pub.publish(center)

if self.detect_flag == False and self.cnt != 0 and self.start_flag == True:
    self.largemodel_arm_done_pub.publish(String(data='garbage_sort_done'))
    self.start_flag = False

if self.detect_flag == False and self.cnt == 0 and self.start_flag == True :
    self.start_flag = False
    print("Did not find the target.")
    self.largemodel_arm_done_pub.publish(String(data='garbage_sort_no_found'))

```

Sort All Garbage on the Desktop

Startup

Input in the text_chat terminal:

```
clean up all garbage on the desktop
```

Press Enter. For the voice version, after waking up the voice module, directly say "Clean up all garbage on the desktop" to the voice module. Wait for the large model to think and reply, as shown below:

```

user input: Clean up all garbage on the desktop
okay@, let me think for a moment.../[INFO] [1775204661.655235971] [text_chat_node]: Decision making AI planning:Call the garbage_sort_all() function
user input: [INFO] [1775204664.100253396] [text_chat_node]: "action": ["garbage_sort('all')"], "response": "I'm going to clean up all the garbage on the desktop, let's get started!"

```

The robotic arm will sort all garbage label blocks on the table:

Task Planning

Call garbage_sort_all() function.

Source code path: `LargeModel_ws/src/largemodel_arm/largemodel_arm/yolov11_garbage.py`,

```

def garbage_sort_all(self):
    Arm.Arm_serial_servo_write6(90,120,0,0,90,30,1000)
    time.sleep(1.0)
    cmd1 = "ros2 run dofbot_yolov11 yolov11_sortation"
    cmd2 = "ros2 run largemodel_arm grasp_obj"

    # Set ROS domain ID to ensure nodes communicate in the same domain
    ros_domain_cmd = "export ROS_DOMAIN_ID=30"

    subprocess.Popen([
        "gnome-terminal",
        "--title=garbage_grasp",
        "--",
        "bash",
        "-c",
        f"{ros_domain_cmd} && {cmd2}; exec bash",
    ])

    # wait for the grasp node to start
    time.sleep(2.0)

```

```

subprocess.Popen([
    "gnome-terminal",
    "--title=garbage_detection_all",
    "--",
    "bash",
    "-c",
    f"{ros_domain_cmd} && {cmd1}; exec bash", # Start garbage detection node
])

while not self.garbage_sort_future.done():
    if self.interrupt_flag:
        self.check_garbage_sort()
        Arm.Arm_serial_servo_write6(90,120,0,0,90,30,1000)
        return
    time.sleep(0.1)

result = self.garbage_sort_future.result()
self.check_garbage_sort_all()
if not self.interrupt_flag:
    self.action_status_pub("garbage_sort_all_done")

```